

Interface Control Document (ICD)

MOTABIQ (مطابق) — Identity Verification API

Document	Interface Control Document — Verification APIs
System	MOTABIQ Verification Middleware
Audience	Integrating service providers (banks) and their backend engineering teams
Interface version	v1
Document version	v2.0 (2026-09-17) — see Appendix B
Transport	HTTPS / REST / JSON
Status	Released

1. Introduction

1.1 Purpose

This document defines the external interface contract between an **integrating service provider** (e.g. a bank) and the **MOTABIQ Verification Middleware**. It specifies the endpoint, authentication, request and response message formats, error codes, and worked samples required to integrate identity verification.

The middleware is a provider-agnostic gateway. It dispatches identity verification to a configured **national identity provider** (e.g. a government civil-registry or national-ID authority), but the integrator-facing contract described here is stable and **independent of the backend provider** — the same request/response shape applies regardless of which country or authority backs a given tenant.

1.2 Scope

This ICD covers the public, bank-facing verification surface only:

- `POST /api/v1/verify/identity` — identity verification.

1.3 Definitions & abbreviations

Term	Meaning
Client ID / Client Secret	Long-lived API credentials issued to the bank tenant. Used for HTTP Basic auth.
Tenant	The bank organisation. All calls are scoped to the tenant resolved from the credentials.

Term	Meaning
Entitlement	The set of fields and operations a tenant's subscription plan permits. Drives response field projection.
Projection	Server-side filtering of the response <code>result</code> object so a tenant only receives the fields its plan entitles.
National Number	The subject's national identification number, as issued by the relevant national-ID authority. The exact format and length are provider/country specific.
Identity provider	The backend national-ID authority/system a tenant is routed to. Provider-specific behaviour is abstracted behind this API.
WSQ	Wavelet Scalar Quantization — a fingerprint image encoding commonly expected for biometrics.
Transaction	A single verification call, recorded with a unique UUID for audit and reconciliation.

2. General interface conventions

2.1 Base URL & environments

The **base URL** is shared together with your **API keys** after onboarding is completed. It is intentionally omitted from this document. Substitute it for `{baseUrl}` in all examples below.

There is no additional context path. The full endpoint path appended to the base URL is:

```
{baseUrl}/api/v1/verify/identity
```

2.2 Transport & encoding

- **Protocol:** HTTPS only (TLS). Plain HTTP is not supported in production.
- **Method:** `POST`.
- **Content type:** `application/json` (request and response).
- **Character encoding:** UTF-8. Response content may be returned in **Latin and/or non-Latin scripts** (for example English and/or Arabic, depending on the provider and the subject's record) and must be handled as UTF-8 end-to-end.
- **Payload encryption:** sensitive request fields (national number + biometrics) are encrypted at the application layer, on top of TLS, using a per-tenant certificate (see §4.2.4). The raw fingerprint is therefore never exposed to TLS-terminating proxies, logs, or storage. **Encryption is mandatory for every verification request.**

2.3 Request correlation — X-Request-Id

Every request **should** carry an `X-Request-Id` header containing a unique identifier (UUID, preferably time-ordered UUIDv7) generated by the caller. It is used for end-to-end tracing across the integrator, the middleware, and the identity provider.

- If omitted, the middleware generates one automatically.
 - The value is echoed back on every response (success and error) in the `X-Request-Id` header and inside error bodies as `requestId`.
 - Always log it on the bank side and quote it when raising support tickets.
-

3. Authentication

3.1 Scheme — HTTP Basic

All calls are authenticated with **HTTP Basic authentication** using the issued Client ID and Client Secret.

```
Authorization: Basic base64(clientId + ":" + clientSecret)
```

Example (illustrative credentials):

```
clientId      = cli_YN-tVFggRQVQ1f0f
clientSecret  = sec_6FVFJkLEQ0Vo4iRm5pLxFocWLp01C7B2rKqitLN0HzI

Authorization: Basic Y2xpX1l0LXRWRmdnUlFWUTFmT2Y6c2VjXzZGVkZKa2xFUTBWbzRpUm01cEx4Rm9jV0xwMD
FDN0IycktxaXRMTk9Iekk=
```

3.2 Credential rules

- Credentials are issued by the platform operator during tenant onboarding and delivered over a secure channel.
- The Client Secret is shown **once** at issuance and stored only as a hash; it cannot be retrieved later. If lost, it must be rotated.
- An **IP allow-list is mandatory** for every credential. The integrator must register their egress IP address(es)/CIDR range(s) during onboarding; calls originating from any other source IP are rejected with **HTTP 403** (`1201 FORBIDDEN`). Notify the platform operator in advance of any change to your egress IPs.
- Credentials may carry an expiry; expired or deactivated credentials are rejected.
- The owning tenant must be in `ACTIVE` status.

3.3 Authentication failures

A missing, malformed, or invalid `Authorization` header — or a rejected credential, IP, or inactive tenant — results in **HTTP 401** with an error body (see §6).

4. Operation: Identity Verification

4.1 Description

Verifies a person against the configured national identity provider by national number, and — when a fingerprint is supplied — returns a biometric match verdict. Without biometrics, the operation returns demographic data only (subject to plan entitlement) and a `NO_VERIFICATION_POSSIBLE` verdict.

Attribute	Value
Method & path	<code>POST /api/v1/verify/identity</code>
Auth	HTTP Basic (§3)
Request media type	<code>application/json</code>
Response media type	<code>application/json</code>
Success status	<code>200 OK</code>

4.2 Request

4.2.1 Headers

Header	Required	Value
<code>Authorization</code>	Yes	<code>Basic base64(clientId:clientSecret)</code>
<code>Content-Type</code>	Yes	<code>application/json</code>
<code>X-Request-Id</code>	Recommended	Unique per call (UUIDv7). Auto-generated if absent.

4.2.2 Body fields

The request body carries a **single field** — the encrypted envelope. All verification requests **must** be encrypted (§4.2.4); unencrypted requests are rejected.

Field	Type	Required	Constraints	Description
<code>encryptedPayload</code>	string	Yes	JWE compact (§4.2.4)	Encrypted envelope whose decrypted content is the PII object below. Built with the tenant certificate (§4.4).

The **decrypted** content of `encryptedPayload` is the following JSON object:

Field	Type	Required	Constraints	Description
<code>nationalNumber</code>	string	Yes	Not blank; max 32 chars	National ID number. Provider-specific format; normalised where applicable.

Field	Type	Required	Constraints	Description
biometrics	object	Yes	—	Fingerprint block. Mandatory so the provider can compute a biometric match verdict.
biometrics.fingerPosition	integer	Yes	1–10	Finger position. 1 = right thumb, 2 = right index, ... 10 = left little.
biometrics.image	string	Yes	base64; must meet §4.2.3	Base64-encoded fingerprint image (WSQ or PNG). Must satisfy the capture-quality constraints in §4.2.3.

Note: inside the encrypted envelope, `nationalNumber` and `biometrics` (both `fingerPosition` and `image`) must be present and meaningful for the provider to compute a verdict. `fingerPosition` outside 1–10 fails validation (HTTP 400).

4.2.3 Fingerprint image quality constraints

When `biometrics` is supplied, the fingerprint image **must** meet the capture-quality constraints below for the provider to compute a reliable match. Images that fail these typically yield `NO_MATCH` or a `400` rejection — enforce them at capture time so the user can re-scan immediately.

Attribute	Requirement
Encoding	Base64 of the raw image bytes; strip any <code>data:</code> URI prefix.
Image format	WSQ (preferred) or PNG . A PNG must be colour type 0 (8-bit greyscale, single plane) — not indexed/palette (colour type 3), not RGB/RGBA, not greyscale+alpha. The provider's matcher rejects multi-plane images ("number of image planes is invalid"), so such images fail even when the print itself is good. Embed the true capture resolution in the PNG <code>phys</code> chunk (500 ppi $\approx$ 19685 px/m).
Resolution	500 ppi (± 10), per FBI/IAFIS Appendix F.
Capture device	FBI-certified single-finger live-scan scanner (FAP 30 minimum; FAP 45/50 recommended), via the certified SDK.
Impression type	Plain (flat) live-scan; exactly one finger per image , matching <code>fingerPosition</code> .
Centring / coverage	Finger centred with the full pad captured; no clipped edges; rotation $\leq 15^\circ$.
WSQ compression	$\leq 15:1$ ratio — do not over-compress.
Quality score	NFIQ 2.0 ≥ 40 recommended; reject low-quality captures and re-scan.
Defects	No latent/partial prints, smudging, moisture halos, or motion blur.

Attribute	Requirement
Size	Within the bound in §7.1 (SHOULD $\leq$ ~256 KB base64; hard transport limit ~2 MB).

These constraints correspond to the bank-side validations **V10–V12** (§7.1). If your scanner SDK cannot report NFIQ, at minimum enforce format, resolution, and single-finger capture, and reject empty/over-compressed images.

Platform-side check (since doc v1.8, codes since v2.0): the platform inspects every image on receipt — container (WSQ/PNG), pixel dimensions, declared resolution, decoded size, WSQ compression ratio, PNG colour depth, blank/coverage tests and the **NFIQ 2 quality score** — before any charge or provider call. An image that fails is rejected with **HTTP 400** and one of two dedicated error codes:

errorCode	error	message	Meaning	Bank action
1003	IMAGE_QUALITY_REJECTED	Fingerprint image quality is not good. Please re-capture the fingerprint.	Blank, partial, smudged or low-quality capture (NFIQ 2 below the minimum).	Show the message to the operator and re-capture . Never resend the same image.
1004	IMAGE_FORMAT_REJECTED	Fingerprint image format is not accepted. Please check the image requirements in the integration guide.	Container, encoding, colour type, size or resolution outside this section.	A software defect on the bank side: log and escalate to the integration team ; re-capturing will not help.

The response body may also carry **imageQuality** (integer 0–100, the NFIQ 2 score) — on quality rejections and on successful verifications — so the capture application can show the operator how far the print is from the threshold. The field is absent when the score was not measured. The image itself is never stored.

Rules applied: container is WSQ or PNG; PNG is 8-bit greyscale colour type 0; 200–2000 px per side; declared resolution, when present, within 490–510 ppi; decoded size $\leq$ 2 MB; WSQ compression $\leq$ 15:1; image not blank (grey-level std dev $\geq$ 10); at least 25 % of 16×16 blocks contain ridge texture; **NIST NFIQ 2 quality score $\geq$ 40** (0–100, measured by the platform; images NFIQ 2 cannot score — blank, partial, fingertip only — are rejected with the reason).

4.2.4 Payload encryption (JWE)

The sensitive payload is encrypted in the caller's system before transmission and decrypted only inside the MOTABIQ gateway. This is **application-layer** encryption layered on top of TLS, and is **mandatory** for every verification request.

Scheme — JWE (JSON Web Encryption), compact serialization:

Parameter	Value
Key management (alg)	RSA-0AEP-256
Content encryption (enc)	A256GCM
Header	must include kid (the certificate's key id from §4.4)
Plaintext	the JSON object { "nationalNumber": "...", "biometrics": { "fingerPosition": 1, "image": "<base64>" } }

Per-tenant keys. Each tenant has its own X.509 certificate. The caller encrypts to the tenant public certificate (retrieved per §4.4); the gateway decrypts with the matching private key, selected by the kid in the JWE header. Keys rotate via kid — re-fetch the certificate when it changes.

Request body — only the envelope is sent; no plaintext PII:

```
POST /api/v1/verify/identity HTTP/1.1
Host: {baseUrl host}
Authorization: Basic Y2xpX1l0LXRWRmdnUlFWUTFmT2Y6c2VjXzZG...
Content-Type: application/json
X-Request-Id: 019dece4-91a1-7877-a37d-ffb20006fcd1

{
  "encryptedPayload": "eyJhbGciOiJSU0EtT0FFUC0yNTYiLCJlbmMiOiJBbmjU2R0NNiIiwia2lkIjoia2V5XzdV
SXZkRkFGRDNBamYyc1MifQ.<encrypted-key>.<iv>.< ciphertext>.<tag>"
}
```

Building the envelope. Generate a random 256-bit content-encryption key (CEK), encrypt the plaintext JSON with A256GCM using a fresh 12-byte IV (with the base64url-encoded protected header as the GCM additional authenticated data), wrap the CEK to the certificate public key with RSA-0AEP-256 , and emit the five-part compact JWE. Standard JWE libraries do this in one call. Reference implementations (Node jose , Python jwcrypto , .NET) and a ready-to-run Postman collection are provided at onboarding — see the *Reference materials* in §10.

A request **without** encryptedPayload , with a malformed envelope, with a kid that is unknown or belongs to another tenant, or with an otherwise undecryptable body is rejected with **HTTP 400 / 403** (1001 BAD_REQUEST / 1201 FORBIDDEN).

4.3 Response (200 OK)

4.3.1 Envelope

Field	Type	Description
transaction	object	Middleware transaction metadata. Always present.

Field	Type	Description
<code>transaction.id</code>	string (UUID)	Unique transaction id (UUIDv7). Quote this for reconciliation and support.
<code>transaction.timestamp</code>	string (ISO-8601 / RFC 3339)	Server time the verification was processed.
<code>transaction.status</code>	string	"OK" for a processed verification.
<code>result</code>	object	The projected canonical verification payload (see below). Fields present depend on the tenant's plan entitlement.

4.3.2 `result` fields (maximum set)

Field	Type	Description
<code>result.transaction.id</code>	string	Provider-side transaction id.
<code>result.transaction.timestamp</code>	string	Provider-side timestamp.
<code>result.verification.verification</code>	string	Verdict: <code>MATCH</code> , <code>NO_MATCH</code> , or <code>NO_VERIFICATION_POSSIBLE</code> (see §5).
<code>result.verification.biometrics.exists</code>	boolean	Whether a biometric comparison was performed.
<code>result.verification.biometrics.score</code>	number	Match score (provider scale, e.g. 0–100).
<code>result.person.nationalNumber</code>	string	Echoed national number.
<code>result.person.demographics.names.arabic</code>	object	Arabic name parts (<code>first</code> , <code>second</code> , <code>third</code> , <code>fourth</code> , <code>last</code> , <code>full</code>).
<code>result.person.demographics.names.latin</code>	object	Latin (transliterated) name parts (same structure).
<code>result.person.demographics.dateOfBirth</code>	string	ISO date-time.
<code>result.person.demographics.nationality</code>	object	e.g. { <code>"id"</code> : <code>"YEM"</code> } .
<code>result.person.demographics.addresses</code>	object	<code>birth</code> , <code>home</code> , <code>work</code> address blocks (when entitled).
<code>result.person.cards</code>	array	Identity document records (when entitled).

The `result` object is loosely typed and tolerant to additive provider changes; integrators should read fields defensively and ignore unknown fields.

4.3.3 Sample response — full-data plan

```
{
  "transaction": {
    "id": "019dece4-91a1-7877-a37d-ffb20006fcd1",
    "timestamp": "2026-05-03T08:11:39.835Z",
    "status": "OK"
  },
  "result": {
    "transaction": { "id": "19dece-152b-7a6c-8e3b", "timestamp": "2026-05-03T11:08:03" },
    "verification": {
      "verification": "MATCH",
      "biometrics": { "exists": true, "score": 92 }
    },
    "person": {
      "nationalNumber": "2132-7404-0424",
      "demographics": {
        "names": {
          "arabic": { "first": "فهد", "second": "صالح", "third": "الخضر",
            "fourth": "سفيان", "last": null,
            "full": "فهد صالح الخضر سفيان" },
          "latin": { "first": "FAHD", "second": "SALEH", "third": "ALKHADHER",
            "fourth": "SUFYAN", "last": null,
            "full": "FAHD SALEH ALKHADHER SUFYAN" }
        },
        "dateOfBirth": "1994-11-17T00:00:00",
        "nationality": { "id": "YEM" },
        "addresses": { "birth": { }, "home": { }, "work": { } }
      }
    }
  }
}
```

4.3.4 Sample response — hit-only plan (verdict only)

```
{
  "transaction": {
    "id": "019dece4-91a1-7877-a37d-ffb20006fcd1",
    "timestamp": "2026-05-03T08:11:39.835Z",
    "status": "OK"
  },
  "result": {
    "verification": {
      "verification": "MATCH",
      "biometrics": { "exists": true, "score": 92 }
    }
  }
}
```

4.3.5 Sample response — no biometrics supplied

```
{
  "transaction": {
    "id": "019dece4-91a1-7877-a37d-ffb20006fcd1",
    "timestamp": "2026-05-03T08:11:39.835Z",
    "status": "OK"
  },
  "result": {
    "verification": {
      "verification": "NO_VERIFICATION_POSSIBLE",
      "biometrics": { "exists": false }
    },
    "person": {
      "nationalNumber": "2132-7404-0424",
      "demographics": { "names": { } }
    }
  }
}
```

4.4 Encryption certificate retrieval

Fetch your tenant's active encryption certificate, used to build the `encryptedPayload` (§4.2.4). Cache it and re-fetch when the `kid` changes (rotation).

Attribute	Value
Method & path	GET /api/v1/crypto/certificate
Auth	HTTP Basic (§3)
Success status	200 OK

Response body:

Field	Type	Description
kid	string	Key id — put this in the JWE <code>kid</code> header.
algorithm	string	Key-management algorithm (<code>RSA-0AEP-256</code>).
encryption	string	Content-encryption algorithm (<code>A256GCM</code>).
certificatePem	string	The X.509 public certificate (PEM) to encrypt to.
fingerprintSha256	string	SHA-256 fingerprint of the certificate, for out-of-band verification.
expiresAt	string (ISO-8601)	Certificate expiry.

```
{
  "kid": "key_7UIvdFAFD3Ajf2sS",
  "algorithm": "RSA-0AEP-256",
  "encryption": "A256GCM",
  "certificatePem": "-----BEGIN CERTIFICATE-----\nMIID...\n-----END CERTIFICATE-----\n",
  "fingerprintSha256": "6E:9C:B3:D1:...",
  "expiresAt": "2028-06-16T07:48:01Z"
}
```

The certificate is also downloadable from the client portal (**API Keys** → **Encryption Certificate**), where it can be rotated. Only the public certificate is ever exposed; the private key never leaves the gateway.

5. Verification verdicts

The `result.verification.verification` field carries the identity outcome. A `200 OK` HTTP status means the call was processed successfully — **it does not by itself mean the identity matched**. Always inspect the verdict.

Verdict	Meaning	Recommended bank action
MATCH	Fingerprint matched the template enrolled at the identity provider.	Accept as a successful biometric identity verification.
NO_MATCH	Fingerprint did not match.	Treat as KYC/identity failure.
NO_VERIFICATION_POSSIBLE	Record found but no biometric template enrolled at the provider for this person, or no biometric was sent. Demographics only.	Fall back to demographic checks; cannot assert biometric identity.

6. Error handling

6.1 Uniform error body

All `4xx` and `5xx` responses use a single, uniform JSON structure:

Field	Type	Description
timestamp	string (ISO-8601)	When the error was generated.
errorCode	integer	Stable numeric MOTABIQ error code (see §6.3).
error	string	Symbolic name of the error code.

Field	Type	Description
message	string	Human-readable detail. For backend errors this includes the upstream provider's own error code (e.g. <code>provider_code 220</code>).
requestId	string	Echo of <code>X-Request-Id</code> for correlation. May be null if none was supplied/generated.
fieldErrors	array	Present only for validation failures; <code>[{ "field": ..., "message": ... }]</code> . Omitted otherwise.
imageQuality	integer	Present only on <code>1003</code> when the platform measured an NFIQ 2 score (0–100). Omitted otherwise.

6.2 Error response headers

In addition to the body, error responses carry:

Header	Description
<code>X-Error-Code</code>	The numeric <code>errorCode</code> .
<code>X-Error-Msg</code>	The <code>message</code> .
<code>X-Request-Id</code>	The correlation id.

6.3 Error code catalog

HTTP	errorCode	error	When it occurs	Caller action
400	1001	BAD_REQUEST	Malformed request (e.g. unparseable JSON); encryptedPayload missing; or an <code>encryptedPayload</code> that is malformed/undecryptable or carries an unknown <code>kid</code> .	Fix the request structure; send <code>encryptedPayload</code> (§4.2.4) built against the current certificate (§4.4).
400	1002	VALIDATION_FAILED	Field validation failed (e.g. blank <code>nationalNumber</code> , <code>fingerPosition</code> out of 1–10) or the provider rejected input (bad ID shape, image refused downstream). <code>message</code> carries the provider's code.	Show user-facing error; let user re-scan. Do not retry identical input.

HTTP	errorCode	error	When it occurs	Caller action
400	1003	IMAGE_QUALITY_REJECTED	The platform measured the fingerprint image and found it unusable: blank, partial, smudged, or NFIQ 2 score below the minimum (§4.2.3). No charge. <code>imageQuality</code> carries the score when measured.	Show the message; re-capture the finger with firm, even pressure. Never resend the same image.
400	1004	IMAGE_FORMAT_REJECTED	The image container or encoding does not meet §4.2.3: not WSQ/PNG, PNG not 8-bit greyscale (colour type 0), wrong size/resolution, WSQ over-compressed. No charge.	Integration defect: log, alert the bank's engineering team , and fix the capture/export code. Re-capturing does not help.
401	1101	UNAUTHENTICATED	Missing/malformed <code>Authorization</code> header.	Add valid Basic auth.
401	1102	INVALID_CREDENTIALS	Wrong client_id/secret, or upstream token rejected.	Verify/rotate credentials.
403	1201	FORBIDDEN	IP allow-list mismatch or upstream permission denied.	Contact platform operator.
403	1202	ENTITLEMENT_DENIED	Subscription does not entitle this operation.	Review plan with operator.
404	1301	NOT_FOUND	National number not found at the provider.	Show "ID not found" to the user.
409	1401	CONFLICT	Resource conflict.	Inspect <code>message</code> .
429	1402	QUOTA_EXCEEDED	Per-minute rate limit or usage quota exhausted.	Back off and retry later.
500	2001	INTERNAL_ERROR	Unexpected server-side error.	Retry with backoff; if persistent, contact support with <code>requestId</code> .
503	2101	CONNECTOR_ERROR	Backend provider returned an error / 5xx / unreachable.	Safe to auto-retry (transient transport only).
503	2102	CONNECTOR_TIMEOUT	Backend timed out.	Safe to auto-retry with backoff.
503	2103	CONNECTOR_UNAVAILABLE	Backend unavailable / circuit open.	Back off and retry later.

Note: all connector failures (2101 / 2102 / 2103) use HTTP **503** so the JSON body always reaches you; distinguish them by `errorCode` . (Earlier versions used 502/504, which the CDN in front of the API replaced with an empty error page.)

Retry guidance: Only `5xx` connector errors (2101 / 2102 / 2103) and `2001` are transient and safe to retry with exponential backoff. `4xx` errors are deterministic — retrying identical input will fail again.

6.4 Sample error responses

400 — validation failure (missing national number):

```
{
  "timestamp": "2026-05-03T08:11:39.948Z",
  "errorCode": 1002,
  "error": "VALIDATION_FAILED",
  "message": "Validation failed",
  "requestId": "019dece4-91a1-7877-a37d-ffb20006fcd1",
  "fieldErrors": [
    { "field": "nationalNumber", "message": "must not be blank" }
  ]
}
```

400 — backend rejected biometrics (provider code surfaced):

```
{
  "timestamp": "2026-05-03T08:11:39.948Z",
  "errorCode": 1002,
  "error": "VALIDATION_FAILED",
  "message": "Provider error 220 · Invalid biometrics.",
  "requestId": "019dece4-91a1-7877-a37d-ffb20006fcd1"
}
```

400 — fingerprint image rejected for quality (platform-side, no charge):

```
{
  "timestamp": "2026-09-17T05:29:05.387Z",
  "errorCode": 1003,
  "error": "IMAGE_QUALITY_REJECTED",
  "message": "Fingerprint image quality is not good. Please re-capture the fingerprint.",
  "requestId": "01a0add7-028c-7b15-a139-76e53b5d0a30",
  "imageQuality": 32
}
```

400 — fingerprint image rejected for format (platform-side, no charge):

```
{
  "timestamp": "2026-09-17T05:29:02.878Z",
  "errorCode": 1004,
  "error": "IMAGE_FORMAT_REJECTED",
  "message": "Fingerprint image format is not accepted. Please check the image requirements in the integration guide.",
  "requestId": "01a0add6-f8b1-7c0e-9a52-3d2f7e1c4b90"
}
```

401 — invalid credentials:

```
{
  "timestamp": "2026-05-03T08:11:39.948Z",
  "errorCode": 1102,
  "error": "INVALID_CREDENTIALS",
  "message": "Invalid credentials",
  "requestId": "019dece4-91a1-7877-a37d-ffb20006fcd1"
}
```

403 — entitlement denied:

```
{
  "timestamp": "2026-05-03T08:11:39.948Z",
  "errorCode": 1202,
  "error": "ENTITLEMENT_DENIED",
  "message": "Subscription does not entitle this operation",
  "requestId": "019dece4-91a1-7877-a37d-ffb20006fcd1"
}
```

404 — national number not found:

```
{
  "timestamp": "2026-05-03T08:11:39.948Z",
  "errorCode": 1301,
  "error": "NOT_FOUND",
  "message": "Resource not found",
  "requestId": "019dece4-91a1-7877-a37d-ffb20006fcd1"
}
```

429 — quota exceeded:

```
{
  "timestamp": "2026-05-03T08:11:39.948Z",
  "errorCode": 1402,
  "error": "QUOTA_EXCEEDED",
  "message": "Usage quota exhausted for tenant",
  "requestId": "019dece4-91a1-7877-a37d-ffb20006fcd1"
}
```

503 — backend connector error:

```
{
  "timestamp": "2026-05-03T08:11:39.948Z",
  "errorCode": 2101,
  "error": "CONNECTOR_ERROR",
  "message": "The verification service could not process the request. Please try again in a few minutes; if the problem persists, contact support and quote the request ID.",
  "requestId": "019dece4-91a1-7877-a37d-ffb20006fcd1"
}
```

7. Integration validation requirements (bank side)

This section is **normative**: the integrating bank **MUST** implement the validations below in its own backend. Server-side validation exists, but failing fast on the client avoids wasted round-trips, prevents unnecessary rejected calls, and protects end users with clear messages. "MUST" = required for go-live; "SHOULD" = strongly recommended.

7.1 Request validations (before sending)

#	Field / item	Rule	Level	On failure (client side)
V1	Authorization header	Present; Basic base64(clientId:clientSecret) ; secret never logged or exposed client-side.	MUST	Do not send; fix configuration.
V2	Content-Type	application/json ; body encoded as UTF-8 .	MUST	Do not send.
V3	X-Request-Id	Generate a unique ID per call (UUID, v7 preferred); log it before sending.	MUST	Generate one; never reuse across distinct calls.
V4	nationalNumber — presence	Non-null, non-blank after trimming whitespace.	MUST	Block; prompt user to enter the ID.
V5	nationalNumber — length	≤ 32 characters.	MUST	Block before sending.
V6	nationalNumber — format	Expected NNNN-NNNN-NNNN (12 digits, dashes). If you collect 12 bare digits, either send them bare (server auto-formats) or format them yourself; reject anything that is not 12 digits / not the dashed form.	SHOULD	Block; prompt re-entry.
V7	nationalNumber — charset	Digits (and dashes) only; strip spaces; reject letters/symbols.	SHOULD	Block; prompt re-entry.

#	Field / item	Rule	Level	On failure (client side)
V8	<code>biometrics</code> — presence	If your KYC flow requires a biometric verdict, the <code>biometrics</code> object must be included; otherwise the verdict will be <code>NO_VERIFICATION_POSSIBLE</code> .	MUST (for biometric KYC)	Capture a fingerprint before submitting.
V9	<code>biometrics.fingerPosition</code>	Required when <code>biometrics</code> is present; integer in 1–10 .	MUST	Block; out-of-range is rejected by the server (400).
V10	<code>biometrics.image</code> — presence	Required when <code>biometrics</code> is present; non-empty.	MUST	Block; re-capture.
V11	<code>biometrics.image</code> — encoding	Valid base64 of the raw bytes; strip any <code>data:</code> URI prefix; no double-encoding.	MUST	Block; fix the export code.
V11a	<code>biometrics.image</code> — container & format	WSQ or PNG only. PNG must be 8-bit greyscale, colour type 0 (never indexed/palette, RGB or grey+alpha). 500 ppi (± 10) declared; 200–2000 px per side; WSQ compression $\leq 15:1$. Export via the scanner SDK's WSQ/greyscale-PNG function, not a generic image library. Violations return <code>400 · 1004</code> .	MUST	Block; fix the capture/export code.
V11b	<code>biometrics.image</code> — capture quality	One finger, centred, full pad, no blank/partial/smudged prints. Enforce the scanner SDK's quality indicator at capture; target NFIQ 2 ≥ 40 when the SDK reports it. Poor captures return <code>400 · 1003</code> with <code>imageQuality</code> .	MUST	Block; operator re-captures.
V12	<code>biometrics.image</code> — size	Keep within a sane bound (SHOULD $\leq$ ~256 KB base64 ; hard server/transport limit is ~2 MB).	SHOULD	Re-capture / re-encode at lower size.
V13	Unknown fields	Send only the documented fields; do not rely on extra fields being processed.	SHOULD	Remove before sending.
V14	Transport	HTTPS only; validate the server TLS certificate (no "trust-all").	MUST	Refuse to send over plain HTTP / on cert error.

#	Field / item	Rule	Level	On failure (client side)
V15	Payload encryption	Encrypt {nationalNumber, biometrics} into encryptedPayload (JWE RSA-OAEP-256 / A256GCM) against the current certificate (§4.2.4, §4.4); send the kid in the JWE header. Encryption is required for every request.	MUST	Fetch/refresh the certificate; build the envelope before sending.

7.2 Response validations (after receiving)

#	Item	Rule	Level
V15	HTTP status	Treat only 200 as processed. Any 4xx / 5xx → parse the uniform error body (§6) and branch on errorCode .	MUST
V16	Verdict, not status	A 200 does not mean the identity matched. Always read result.verification.verification and branch on MATCH / NO_MATCH / NO_VERIFICATION_POSSIBLE (§5).	MUST
V17	Field projection tolerance	The result shape depends on the plan — fields outside entitlement are absent , not null. Read defensively; never assume person / demographics exist; ignore unknown fields.	MUST
V18	transaction.id	Persist it against your record for reconciliation and support.	MUST
V18a	Image rejection codes	Branch on errorCode : 1003 → show message and re-capture (never resend the same image); 1004 → log, raise an integration defect, do not prompt the user to retry. When imageQuality is present, you MAY display it (e.g. "quality 32 of 100, minimum 40") to guide the operator.	MUST
V19	X-Request-Id round-trip	Confirm the response X-Request-Id / error requestId matches what you sent; log both sides.	SHOULD
V20	UTF-8 / scripts	Decode and store names as UTF-8; correctly render both Latin and non-Latin scripts (e.g. Arabic RTL).	MUST
V21	Retry policy	Retry only 5xx connector errors (2101 / 2102 / 2103) with exponential backoff; back off on 429 ; never retry 4xx (deterministic).	MUST
V22	Idempotency on timeout	On an ambiguous timeout, do not blindly resubmit — reconcile by X-Request-Id / transaction.id first to avoid duplicate transactions.	SHOULD
V23	PII / biometric hygiene	Never log nationalNumber or biometrics.image in plaintext; mask in logs; store per your data-protection obligations.	MUST

7.3 Timeouts

Set a client read timeout of **at least 30 seconds** to accommodate upstream provider latency plus internal retries.

8. Operational considerations

8.1 Rate limits & quota

Each credential is subject to a per-minute rate limit and a usage quota. Exceeding either returns **HTTP 429 / 1402 QUOTA_EXCEEDED** ; the transaction is recorded as rejected. Implement client-side backoff.

8.2 Resilience

The middleware applies circuit-breaker and retry policies to the backend connector. Transient transport failures are retried automatically before a `5xx` is surfaced. `4xx` results from the backend are never retried internally.

8.3 Transactions & audit

Every call is recorded as a transaction with its `transaction.id` , status, and correlation `X-Request-Id` for audit and reconciliation. Rejected/error transactions are recorded with their error code.

8.4 Timeouts

Callers should set a client read timeout of at least 30 seconds to accommodate upstream provider latency plus internal retries.

9. Acceptance criteria

The integration is accepted (UAT sign-off) when **every "Must" criterion below passes** against the staging environment. Each row is an independent, repeatable test. "Must" criteria are mandatory for go-live; "Should" criteria are tracked but non-blocking. A companion spreadsheet (`M0TABIQ_Verification_Acceptance_Criteria.xlsx`) is provided to record pass/fail sign-off per criterion. Test identities are provided separately during onboarding.

ID	Scenario	Expected result	Level
AC-01	Valid credentials + a valid test NID (provided separately) + valid fingerprint.	<code>200 OK</code> ; <code>transaction.id</code> present (UUID); <code>transaction.status = "OK"</code> ; verdict returned.	Must
AC-02	Missing / malformed <code>Authorization</code> header.	<code>401</code> ; <code>errorCode 1101 UNAUTHENTICATED</code> .	Must

ID	Scenario	Expected result	Level
AC-03	Wrong client_id / client_secret.	401 ; errorCode 1102 INVALID_CREDENTIALS .	Must
AC-04	Valid NID + valid fingerprint of the enrolled person.	200 ; verdict MATCH ; biometrics.score present; bank treats as identity pass.	Must
AC-05	Valid NID + fingerprint of a different person.	200 ; verdict NO_MATCH ; bank treats as identity failure (not accepted).	Must
AC-06	Blank nationalNumber .	400 ; errorCode 1002 VALIDATION_FAILED ; fieldErrors lists nationalNumber ; bank shows error and does not retry.	Must
AC-07	fingerPosition out of range (e.g. 11).	400 ; errorCode 1002 ; bank blocks this client-side per V9 before sending.	Must
AC-08	Corrupt / invalid biometric image.	400 ; errorCode 1002 (message includes the upstream provider code); user re-scans; no retry of identical input.	Must
AC-09	National number not found at the provider.	404 ; errorCode 1301 NOT_FOUND ; bank shows "ID not found".	Must
AC-10	Rate limit / usage quota exceeded.	429 ; errorCode 1402 QUOTA_EXCEEDED ; bank backs off and retries later.	Must
AC-11	Backend provider error / unavailable.	5xx (2101 / 2102 / 2103); bank performs safe exponential-backoff retry; never retries 4xx .	Must
AC-12	Call from an unregistered source IP.	403 ; errorCode 1201 FORBIDDEN (mandatory IP allow-list, §3.2).	Must
AC-13	X-Request-Id correlation.	The ID the bank sends is echoed in the response X-Request-Id header and in error requestId ; bank logs both sides.	Must
AC-14	Script rendering end-to-end.	Returned names (Latin and/or non-Latin, e.g. Arabic) are stored/displayed correctly as UTF-8 (RTL where applicable), no mojibake.	Must
AC-15	Transport security.	All calls over HTTPS; server certificate validated; plain HTTP refused by the client.	Must
AC-16	Projection tolerance.	Integration parses a response with entitlement-trimmed fields (some absent) without error.	Must
AC-17	PII / biometric hygiene.	Review bank logs: no nationalNumber or biometric image stored/logged in plaintext (per V23).	Must
AC-18	Reconciliation.	transaction.id is persisted and matches the corresponding transaction in the MOTABIQ portal.	Should

ID	Scenario	Expected result	Level
AC-19	Timeout handling.	Client read timeout $\geq$ 30 s; slow responses handled gracefully without duplicate submission (per V22).	Should
AC-20	Low-quality fingerprint (light pressure, partial or smudged capture).	400 ; errorCode 1003 IMAGE_QUALITY_REJECTED ; imageQuality present when scored; operator is prompted to re-capture; the same image is not resent; no wallet charge.	Must
AC-21	Wrong image format (e.g. PNG exported as indexed/palette or RGB, or WSQ over-compressed).	400 ; errorCode 1004 IMAGE_FORMAT_REJECTED ; bank logs the event as an integration defect and does not prompt the user to retry; no wallet charge.	Must
AC-22	Good fingerprint — quality score returned.	200 ; response carries imageQuality (0–100) alongside the verdict; bank stores it with transaction.id .	Should

Sign-off: record pass/fail per AC ID with the `X-Request-Id` and `transaction.id` used, attach to the UAT report, and obtain joint approval from the bank and MOTABIQ before enabling production credentials.

10. Reference materials

- **OpenAPI / Swagger UI:** `{baseUrl}/swagger-ui.html`
- **OpenAPI spec (service provider group):** `{baseUrl}/v3/api-docs/service-provider`
- **Postman collection:** `postman/Sannad-Verification-API.postman_collection.json` (auto-generates `X-Request-Id` , stores `lastTransactionId`).
- **Encrypted Postman collection:** `postman/MOTABIQ-Verify-Encrypted.postman_collection.json` — fetches the certificate and builds the `encryptedPayload` automatically (§4.2.4).
- **Payload encryption guide (Node / Python / .NET):** `docs/PII_ENCRYPTION.md` .

Appendix A — Quick integration checklist

1. Obtain Client ID / Client Secret over a secure channel.
2. Register your egress IP(s)/CIDR(s) in the **mandatory** IP allow-list, and confirm network egress can reach the base URL (shared with your API keys after onboarding).
3. Build the `Authorization: Basic` header from `clientId:clientSecret` .
4. Generate a UUIDv7 `X-Request-Id` per call and log it.
5. Send a test call to a valid test NID (provided separately during onboarding); expect `200 OK` .
6. Enforce the fingerprint image quality constraints (§4.2.3) at capture time, and handle `1003` (re-capture) and `1004` (integration defect) separately.
7. Handle the verdict (`MATCH` / `NO_MATCH` / `NO_VERIFICATION_POSSIBLE`) — not just the HTTP status.
8. Implement backoff for `429` and retry only for `5xx` connector errors.
9. Rotate the Client Secret before going to production.

Appendix B — Change log

Version	Date	Notes
v1	2026-06-04	Initial ICD for <code>POST /api/v1/verify/identity</code> .
v1.1	2026-06-04	Added §7 Integration validation requirements (bank side) and §10 Acceptance criteria; renumbered Operational/Test data/Reference sections.
v1.2	2026-06-04	Made the document provider-agnostic (no longer Yemen/MOI specific); added §4.2.3 fingerprint image quality constraints; made the IP allow-list mandatory; clarified UTF-8 covers Latin and non-Latin scripts; removed billing/payment details (technical scope only).
v1.3	2026-06-10	Made <code>biometrics</code> mandatory; removed the no-biometrics sample request; removed the Test data section (provided separately at onboarding); acceptance criteria sign-off tracked in a companion <code>.xlsx</code> ; renumbered Reference materials.
v1.4	2026-06-17	Added end-to-end payload encryption : per-tenant JWE (<code>RSA-0AEP-256</code> + <code>A256GCM</code>) of the PII via <code>encryptedPayload</code> and the certificate-retrieval endpoint <code>GET /api/v1/crypto/certificate</code> (§4.4); added request validation V15 and encryption error guidance (§6.3).
v1.5	2026-06-17	Made encryption mandatory — removed the legacy plaintext request shape and all dual-accept/enforcement wording. The request body is now solely <code>encryptedPayload</code> (§4.2.2); the encryption scheme + sample is §4.2.4.
v1.6	2026-06-17	Removed all environment URLs; the base URL is shared with the API keys after onboarding. Examples use a <code>{baseUrl}</code> placeholder (§2.1).
v1.7	2026-09-13	Connector failures <code>2101</code> / <code>2102</code> now return HTTP 503 (previously 502/504) so the JSON error body is never replaced by the CDN's generic error page; <code>2103</code> unchanged. No change to <code>errorCode</code> values or body shape.
v1.8	2026-09-15	Platform-side structural validation of <code>biometrics.image</code> (§4.2.3): failing images return <code>400 · 1002</code> with a <code>Fingerprint image rejected: ...</code> message before any charge.
v1.9	2026-09-17	Platform-side image check now returns two plain messages (quality / format) instead of technical detail; NFIQ 2 score (≥ 40 , bank-specific thresholds possible) measured server-side; optional <code>imageQuality</code> field on success and quality-rejection responses.
v2.0	2026-09-17	New error codes <code>1003 IMAGE_QUALITY_REJECTED</code> and <code>1004 IMAGE_FORMAT_REJECTED</code> for platform-side image rejections (previously <code>1002</code>); <code>imageQuality</code> documented in §6.1; bank validations V11 split into V11/V11a/V11b and V18a added; acceptance criteria AC-20 – AC-22 added.